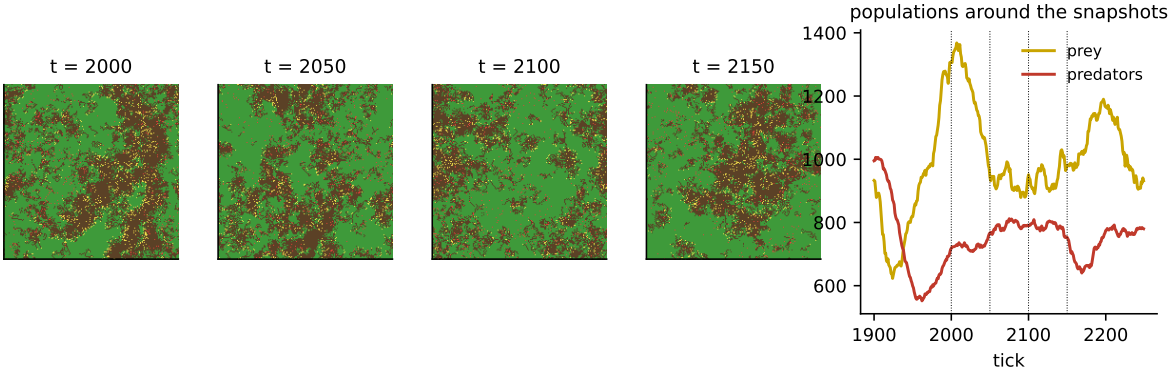
Predators, Prey and Feedback

A hands-on guide to dynamic systems with the Lotka ecosystem simulator

For first-year Systems & Control students



Key idea: what this guide is about

A *dynamic system* is anything whose future depends on its present: a cruise-controlled car, a thermostat-heated room, a bank balance, an epidemic... and an ecosystem. In this guide you will *play* with a simulated ecosystem of vegetation, prey and predators, and use it to discover, one by one, the ideas that the rest of your Systems & Control course is built on: **state, feedback, equilibrium, stability, oscillation, delay, nonlinearity, models** and finally **control**. Very little mathematics is needed; curiosity and the simulator are enough.

Contents

1 Getting started 3

2 What is a dynamic system? 3

3 Feedback loops 4

4 Equilibrium 4

5 Stability: does it come back? 5

6 Oscillations and delays 6

7 Nonlinearity and tipping points 7

8 Models of the system 8

9 Control: steering the ecosystem **9**

10 Letting the computer search: optimisation **10**

11 Summary **11**

Exercises **11**

1 Getting started

You need the two browser pages of the Lotka project (your instructor will give you the address, usually `http://127.0.0.1:8765`):

- **Simulator** (`/`): the grid of coloured pixels, live charts and all the knobs.
- **Command Center** (`/evolve`): lets the computer search for parameter sets by itself (Section 10).

Reading the grid. Every pixel is a patch of land. **Green** = vegetation, **brown** = bare soil that is regrowing, **yellow** = a prey animal, **red** = a predator. The world wraps around: leaving on the right brings you back on the left.

The rules (one *tick* = one time step):

1. Eaten vegetation grows back after `veg_regrow_time` ticks.
2. Animals move one cell. Prey drift towards grass and away from predators; predators drift towards prey.
3. A prey next to a predator is eaten (a predator eats at most one per tick by default).
4. Prey eat the grass under and around them. A prey that goes `prey_starve_time` ticks without enough food dies; a predator dies after `predator_starve_time` ticks without a meal.
5. Fed animals may have one offspring in a free neighbouring cell.

The controls. Play/Step run time forward; Apply & reset restarts with the parameters on the left; the **Preset** menu loads ready-made scenarios; the **Shock** buttons suddenly remove or add animals; the charts on the right show populations over time, the *phase portrait* (prey vs predators) and births/deaths per tick. The panel at the bottom compares the simulation with mathematical models (Section 8).

Try it: first contact

Load the preset *Stable coexistence* and press Play. Watch the grid for a minute. Then press Pause and use Step a few times. Write down three things you notice.

2 What is a dynamic system?

Key idea: state

The **state** of a system is the information you need *now* to work out what happens *next*. A rule (here: the five rules above) takes the state at tick t and produces the state at tick $t + 1$:

$$\text{state}(t+1) = \text{rule}\big(\text{state}(t),\ \text{inputs}(t)\big).$$

For our ecosystem the *true* state is enormous: the content of every cell, plus every animal's hunger and age — more than a hundred thousand numbers. Engineers almost never work with the full state. Instead we pick a few **state variables** that summarise it well: the number of prey x, the number of predators y and the amount of vegetation v. The charts show exactly these three numbers.

An **input** is something we can push on from outside (later: harvesting), an **output** is what we measure (the counts in the cards at the top), and a **disturbance** is a push we do not control (the Shock buttons will play that role).

Think about it

Is the population of prey next tick fully decided by today's population of prey? Why do you also need to know how many predators there are? What about vegetation? This is why a dynamic system usually needs *several* state variables.

3 Feedback loops

Look at how the three variables influence each other (Fig. 1). An arrow marked + means "more of A leads to more of B"; − means "more of A leads to less of B".

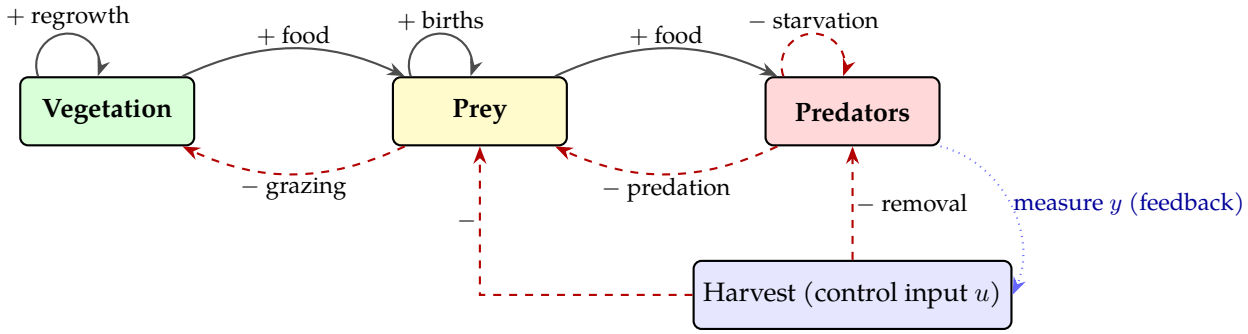


Figure 1: Causal-loop diagram of the ecosystem. Solid arrows reinforce (+), dashed arrows oppose (−). The harvest box is the control input used in Section 9.

Key idea: two kinds of loop

Reinforcing (positive) loops amplify change: more prey → more births → even more prey. On their own they give explosive growth.
Balancing (negative) loops oppose change: more prey → more predators → fewer prey. They push the system back. Follow the loop prey → predators → prey: one + and one −, so the overall sign is −: it is balancing.

Try it: break a loop

Load *Predator starvation*: predators die after only 3 hungry ticks, so the predation loop is cut almost immediately. What happens to the prey? What limits them now? (Hint: look at the green.) Then load *Prey overpopulation* (no predators at all, fast breeding). Describe what the reinforcing birth loop does at first and which balancing loop eventually stops it.

4 Equilibrium

Key idea: equilibrium

An **equilibrium** is a state that does not change on its own: everything that flows in is balanced by what flows out. For a population: *births* = *deaths*.

Try it: find the balance

Load *Stable coexistence* and run for 1000 ticks. Look at the *Events per tick* chart: compare prey births with prey eaten + prey starved. Look at the cards: the counts wobble but stay around the same values. Roughly what are the equilibrium numbers of prey and predators?

Think about it

At equilibrium the *numbers* are steady, yet animals are still being born and dying every tick. An equilibrium is a balance of flows, not an absence of activity — exactly like a bathtub whose water level stays constant because the tap and the drain run at the same rate.

A surprising consequence (Lotka and Volterra noticed it in the 1920s): the equilibrium number of *prey* is set mostly by the *predators'* properties, and vice versa. Try it: in *Stable coexistence* make predators hungrier-tolerant (`predator_starve_time` from 12 to 20) and check which equilibrium moves most.

5 Stability: does it come back?

Key idea: stability

Push a ball at the bottom of a bowl: it rolls back. That equilibrium is **stable**. Balance it on top of an upturned bowl: the slightest push and it is gone — **unstable**. To test stability you *perturb* the system and watch whether it returns.

Try it: shock the ecosystem

In *Stable coexistence*, run to tick 1000. Press −50% pred. and keep running.

1. What happens to prey right after the shock? And a little later?
2. Does the system return to where it was? Roughly how many ticks does it take?
3. Does it return smoothly, or does it overshoot and swing around before settling?

Repeat with +100% prey.

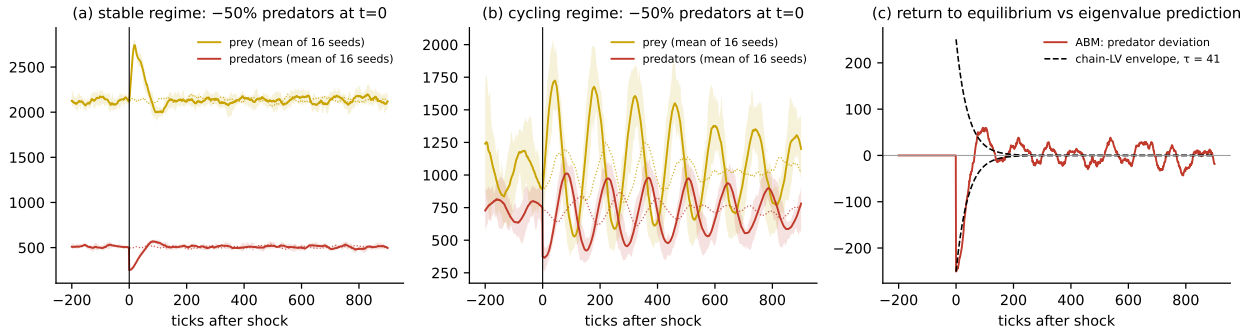


Figure 2: Shock experiments averaged over many runs. (a) In the stable regime the populations swing back to equilibrium in a few hundred ticks, with a decaying oscillation. (b) In the cycling regime the shock just shifts the timing of the cycles. (c) The size of the deviation shrinks roughly like an exponential envelope; its time constant can be predicted from a mathematical model (Section 8).

A stable system that returns with decaying swings is called **underdamped** — the same word you will meet for a car suspension or a control loop. The time it takes for the swings to shrink by a factor $e \approx 2.7$ is the **time constant** τ.

Think about it

Is there a shock big enough that the system does *not* come back? Try [−50% prey] three times in quick succession. A stable equilibrium usually has a limited *basin of attraction*: small pushes are corrected, very large ones can tip the system somewhere else (e.g. extinction).

6 Oscillations and delays

Predator–prey systems are famous for **cycles**: prey boom, then predators boom, prey crash, predators starve, and around it goes.

Try it: make it cycle

Load *Lotka-Volterra cycles* (identical to the stable case except that predators survive 30 ticks without food instead of 12). Run 2000 ticks.

1. Measure the **period**: ticks from one prey peak to the next.
2. Which peaks first, prey or predators? By how much?
3. Look at the phase portrait. Which way does the curve turn?

Key idea: delay causes overshoot

Why does a longer starvation time make the system oscillate? Predators react to food shortage only after a **delay**: they keep eating even when prey are already scarce, so prey are pushed far below the balance point, and then predators die all together. A balancing loop with a long delay overshoots — like a shower whose temperature responds slowly to the tap: you keep turning, overshoot, turn back, overshoot again...

Reading a phase portrait. Instead of plotting x and y against time, plot y against x. Time becomes a path. A point that stays put is an equilibrium; a closed loop is a cycle; an inward spiral is a damped oscillation. In predator–prey systems the loop turns anticlockwise: prey grow (move right), then predators grow (move up), then prey fall (left), then predators fall (down).

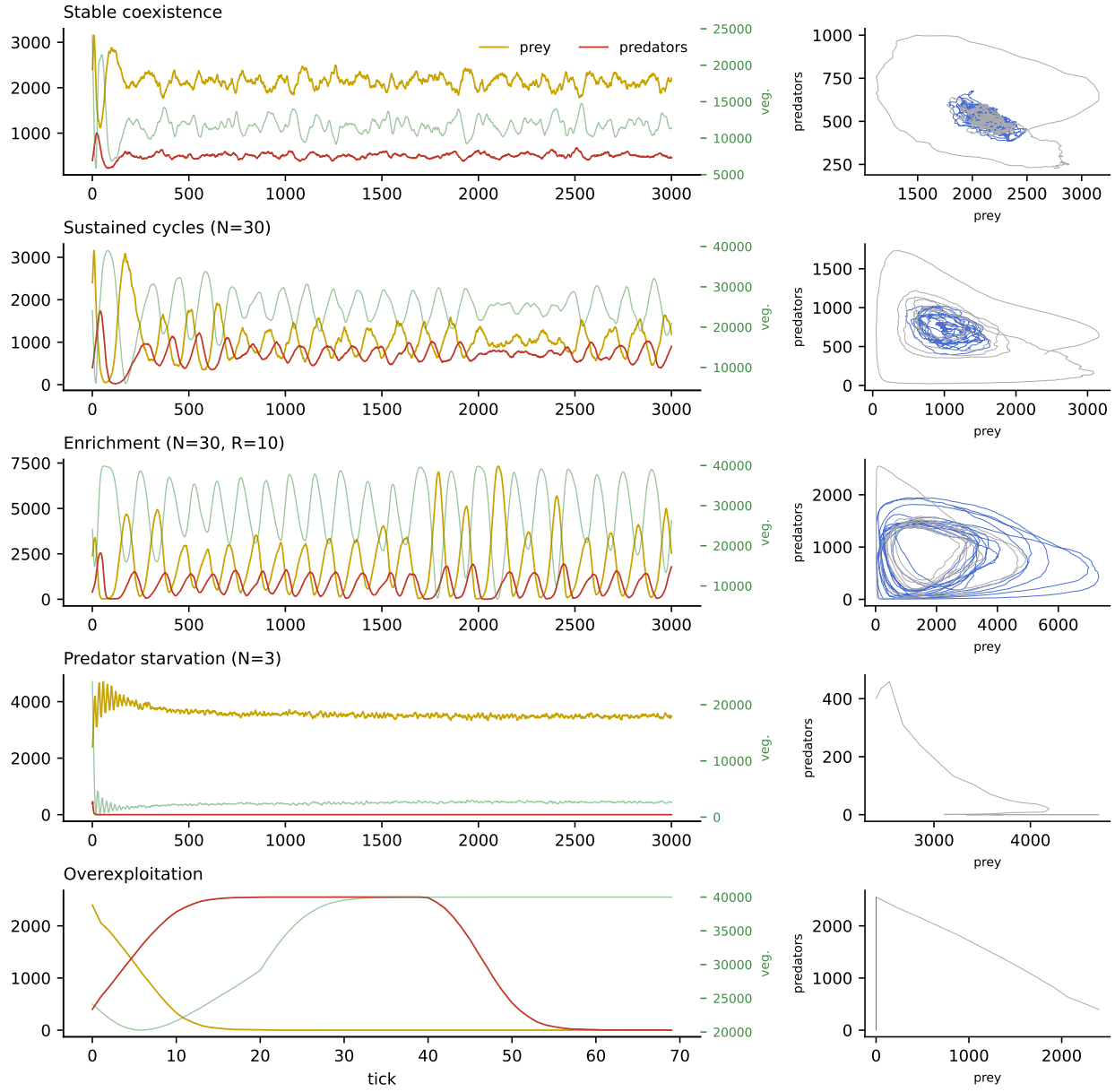


Figure 3: Five regimes produced by the same rules with different parameters. Left: populations over time (green, right axis: vegetation). Right: phase portraits, prey on the horizontal axis and predators on the vertical (grey = start, blue = later). A cycle is a loop; a stable equilibrium is a small blob.

7 Nonlinearity and tipping points

Key idea: nonlinear means "not proportional"

In a *linear* system, doubling the push doubles the response. Our ecosystem is **nonlinear**: a small change can do almost nothing, or everything.

Try it: the paradox of enrichment

Compare *Lotka-Volterra cycles* with *Paradox of enrichment*, where vegetation grows back twice as fast. More food should be good for everybody, right? Compare the size of the swings and the lowest populations reached. Ecologists call this the *paradox of enrichment* (Rosenzweig, 1971).

Try it: overexploitation

Load *Overexploitation*. Predators here are super-hunters. What happens to them in the end, and why? A controller that is "too good" at its job can destroy the very thing it depends on.

8 Models of the system

In 1925–26 Alfred Lotka and Vito Volterra described predators and prey with two short equations. In words:

$$\text{change of prey per tick} = \underbrace{a\,x}_{\text{births}} - \underbrace{b\,x\,y}_{\text{eaten}}$$

$$\text{change of predators per tick} = \underbrace{c\,b\,x\,y}_{\text{births from food}} - \underbrace{g\,y}_{\text{deaths}}$$

The term $b\,x\,y$ says: meetings between prey and predators are proportional to how many of each there are. Setting both changes to zero (the equilibrium!) gives, with one line of algebra,

$$x^* = \frac{g}{c\,b}, \qquad y^* = \frac{a}{b},$$

which is the "surprising consequence" of Section 4: prey equilibrium depends only on predator numbers g, c, b.

Try it: model versus simulation

Run *Lotka-Volterra cycles* for 1500 ticks, then in the bottom panel choose a model and press Fit. The program measures births, deaths and kills in the simulation, estimates a, b, c, g, and draws the model's prediction (dashed) on top of the simulation (solid). Try the different models in the menu.

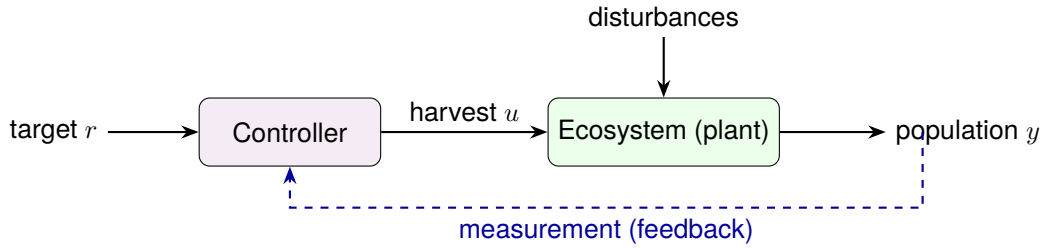
1. Does the model get the *average* populations right?
2. Does it get the *wobbles* right? The period?
3. Choose *Mean-field of rules*: a model built from the rules with no fitting at all, that ignores where animals are. How does it compare?

Key idea: all models are wrong, some are useful

The equations get the averages remarkably right, but they predict that the swings die out, while the simulation keeps wobbling: random births and deaths keep "kicking" a system that, on its own, would settle. The model also ignores *space*: in the simulation predators eat the prey around them and must then travel, which slows predation down by a factor of about three compared with a well-mixed world, and makes the ecosystem much more stable. Choosing what to leave out of a model is the central skill of a systems engineer.

9 Control: steering the ecosystem

So far we only watched. Now we *act*. Our actuator is **harvesting**: removing animals each tick (think of fishing or culling). In control language:



The **Control** group of parameters (bottom of the left panel) offers three controllers:

constant effort remove a fixed *fraction* of the population each tick (open loop);
constant quota remove a fixed *number* of animals each tick (open loop);
feedback (P-control) measure the population y and remove $K_p\,(y - r)$ animals each tick when y is above the target r (closed loop).

`harvest_species` selects prey, predators or both, and `harvest_start` the tick at which the controller switches on, so that you can see the system before and after.

Try it: open loop – quota versus effort

Start from *Stable coexistence*, harvest prey, `harvest_start` = 500.

1. Constant quota of 20 prey per tick. Then 60. Then 80. What happens?
2. Constant effort of 0.01, 0.03, 0.06. Compare.

Which strategy is more dangerous, and why? (Hint: what does each one do when the population is already low?)

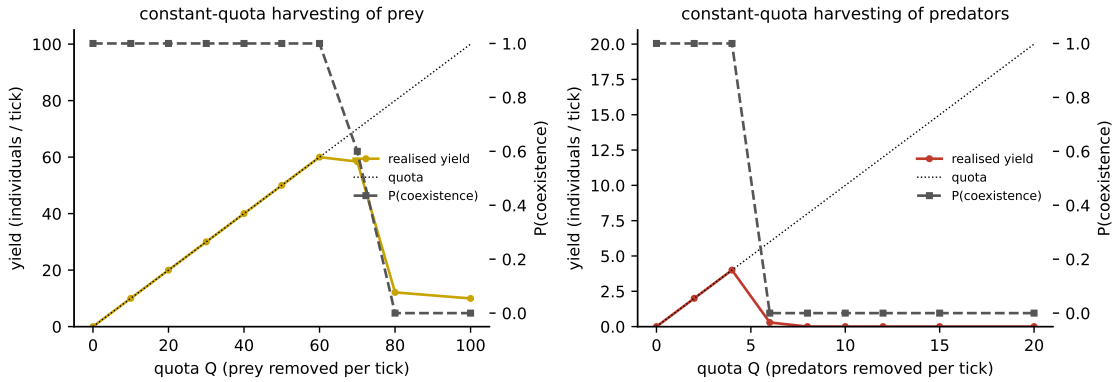


Figure 4: Constant-quota harvesting: up to a threshold the yield equals the quota and both species survive; just beyond it the prey population collapses and so does the ecosystem (grey: probability of coexistence).

Control corner: why the quota tips over

A quota is *blind*: it takes the same number whether there are 3000 prey or 30. When prey are scarce it takes a larger and larger *fraction* of them — a reinforcing loop towards extinction. Constant effort takes a fixed fraction, so it automatically takes less when the population is low: it has a built-in balancing loop. Real fisheries learned this lesson the hard way.

Try it: closed loop – taming the cycles

Load *Lotka-Volterra cycles*. Set *feedback* control on predators with target 400 and `harvest_start` = 1500. Try gains $K_p = 0.02,\ 0.2,\ 1$. Observe:

1. Do the cycles shrink after the controller switches on?
2. Does the predator population sit exactly on the target?
3. What does the controller do when predators are *below* the target?

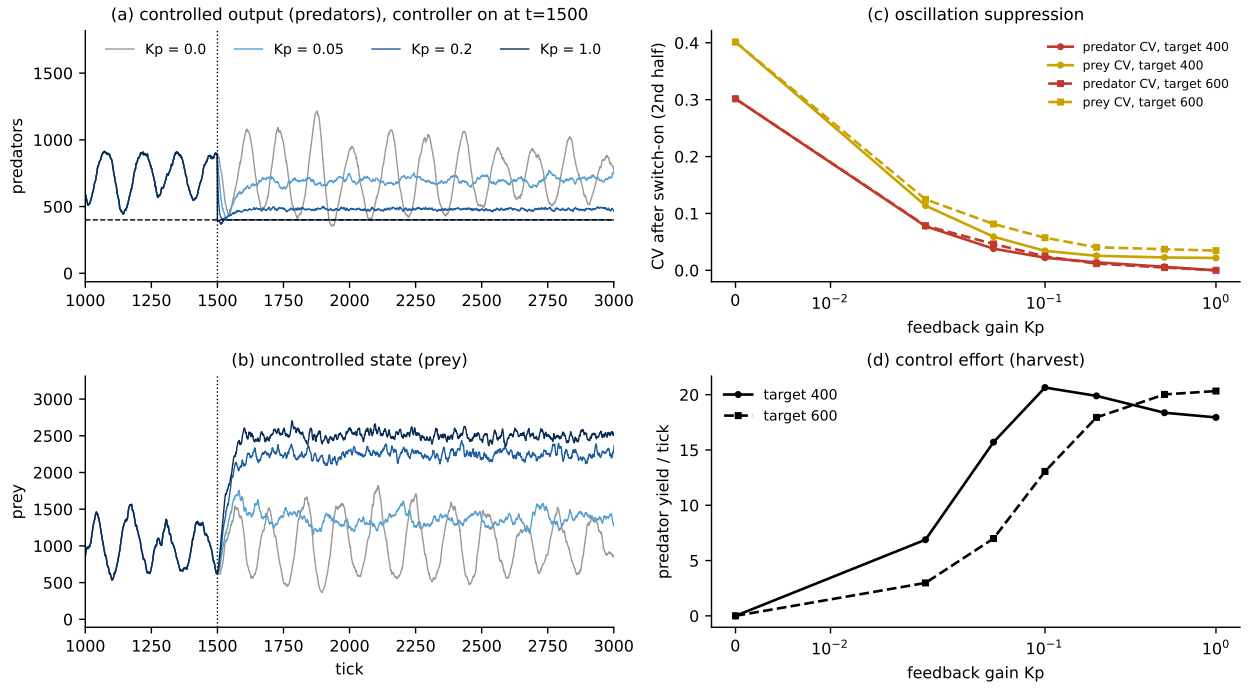


Figure 5: Feedback control of predators in the cycling regime (switched on at $t = 1500$). Higher gains cap the predator peaks and the prey cycles shrink too, because the balancing loop through the predators is no longer delayed. (c) Oscillation size versus gain, (d) the harvest the controller has to take.

Control corner: what you just discovered

- **Feedback beats open loop** at rejecting disturbances: the controller reacts to what actually happens.
- **Actuator limits**: harvesting can only *remove* animals. When the population is below target the controller can do nothing — the actuator *saturates*. Real actuators (motors, valves, heaters) always have limits.
- **Steady-state error**: a proportional controller only acts when there is an error, so the population hovers *above* the target rather than on it.
- **Gain trade-off**: a small gain barely helps; a large one acts hard and takes a lot of harvest.

10 Letting the computer search: optimisation

With 20+ parameters, trying combinations by hand is hopeless. The **Command Center** runs a **genetic algorithm**: each candidate parameter set is a "DNA"; many candidates are simulated, scored by a **fitness** function, and the best are combined and mutated to form the next generation.

Try it: run an optimisation

Open the Command Center. Choose the objective *coexistence*, population 16, generations 8, ticks 800, grid size 60, and untick every gene except `predator_starve_time`, `predator_repro_prob` and `prey_repro_prob`. (A run this size takes a few minutes.) Press Start run and watch the fitness chart. When it finishes, open the best parameter set in the simulator. Is it really a robust ecosystem?

Think about it: you get what you measure

In our first large optimisation, the computer "discovered" that the easiest way to make both species survive was to make predators that never reproduce and never die of old age — a handful of immortal predators living forever among a sea of prey. It satisfied the fitness function perfectly and was ecologically meaningless. We fixed it by giving animals a finite lifespan and requiring both species to keep reproducing. In control and optimisation, *writing the right objective* is the hardest and most important step.

11 Summary

Concept	In the ecosystem	In engineering
State	prey, predators, vegetation	position & speed of a car
Feedback loop	predators limit prey	thermostat
Equilibrium	births = deaths	room at set temperature
Stability	returns after a shock	suspension after a bump
Delay	predators starve slowly	slow shower tap
Oscillation	boom-and-bust cycles	under-damped motor
Nonlinearity	enrichment destabilises	saturating amplifier
Model	Lotka–Volterra equations	transfer function
Open-loop control	fixed quota / effort	toaster timer
Closed-loop control	harvest $\propto$ (population $-$ target)	cruise control
Optimisation	genetic algorithm	controller tuning

Exercises

E1. **Draw your own loop diagram** for a room with a heater controlled by a thermostat. Identify the balancing loop. Where would a delay come from?

E2. In *Stable coexistence*, change `predator_starve_time` in steps 8, 12, 16, 20, 25, 30. For each, note whether the system is steady or cycling. At roughly which value does the behaviour change?

E3. Measure the time constant of the return after a -50% predator shock (time for the deviation to drop to about one third). Do it twice with different seeds. Why are the answers different?

E4. Make the grid smaller (`width` = `height` = 50) in *Lotka-Volterra cycles*. Run five seeds. How often does an ecosystem collapse? Explain using the idea of random "kicks".

E5. Find the largest constant quota of prey that the stable ecosystem tolerates for 3000 ticks. Then find the effort that gives the same average yield. Compare how close each is to collapse.

E6. With feedback control on predators, plot (on paper) the average predator population against the gain K_p for target 400. Why is it always above 400?

E7. Use the Command Center objective *yield* with genes `harvest_setpoint` and `harvest_gain` (fix `harvest_mode`=3 and `harvest_species`=1). What controller does the computer find? Is it a sensible design?

Hints

E2: there is no sharp switch — the wobbles grow steadily with N (in our runs the prey CV goes from about 0.06 at $N = 12$ to 0.36 at $N = 30$); below about $N = 10$ predators go extinct and from about $N = 50$ they eat themselves out of food. **E3**: randomness — each run is one sample of a noisy system; averaging many runs (as in Fig. 2) gives a clean answer. **E4**: in a small world the random fluctuations are large compared with the populations, so a trough can hit zero (in our runs a 40×40 cycling world never survived 3000 ticks, 100×100 survived 4 times in 5, and 140×140 or larger always). **E5**: in our runs a quota of 60 prey/tick was tolerated and 70 already killed 2 of 5 ecosystems; the quota fails abruptly, effort degrades gracefully. **E6**: a proportional controller needs an error to act.

The Lotka simulator, the experiments behind every figure and the companion research paper are available at `https://github.com/hhpmacedo/lotka`.